Appello 1 Luglio 2016



Politecnico di Milano
Anno accademico 2014-2015

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Ghezzi	☐ Mottola	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

Esercizio 1:
Esercizio 2:
Esercizio 3:
Esercizio 4:

Esercizio 1

Si consideri la classe `QuasiRetta` che contiene un insieme di punti che approssimano una retta.

Il costruttore della classe riceve come parametri due punti (definiti dalla classe `Punto`) e un numero reale `approx`. Si assuma già definita la classe Punto, immutabile e con un metodo `boolean equals(Punto p)` definita opportunamente. Questi due punti definiscono univocamente una retta. Il significato di `approx` viene chiarito nel seguito.

Dopo la creazione, ulteriori punti possono essere aggiunti mediante il metodo pubblico `void aggiungi (Punto p)` fornito dalla classe. Questo metodo aggiunge alla retta il punto `p` purche' questo disti dalla retta non piu' di `approx`. La distanza di un punto dalla retta viene calcolata da un metodo pubblico `float dist(Punto p)` che calcola la distanza del punto `p` dalla retta. Esistono anche un metodo `void elimina(Punto p)` che elimina il punto passato come parametro, se questo esiste in `QuasiRetta` e un metodo `boolean appartiene(Punto p)` che verifica se il parametro appartiene a `QuasiRetta`. Si suppone che la classe esporti un attributo pubblico e finale `APPROX` il cui valore e' l'approssimazione che viene passata alla classe con il costruttore. Da ultimo, il metodo `int totBuoni()` restituisce il numero di punti di `QuasiRetta` che possono essere considerati appartenenti alla retta, in quanto la loro distanza e' inferiore al valore di `APPROX` diviso per 1000.

Si chiede di specificare formalmente in termini di pre e post condizioni JML, definendo eventualmente altri metodi pubblici puri che ritenete utili, le seguenti operazioni:

1. il costruttore `QuasiRetta(Punto p1, Punto p2, float approx);`
2. il metodo `void aggiungi(Punto p) throws IllegalPointException`, dove l'eccezione viene lanciata se la distanza del punto dalla retta eccede l'approssimazione richiesta;
3. il metodo `void elimina(Punto p);`
4. il metodo `int totBuoni().`

Si supponga di voler definire anche un metodo `ArrayList<Punto> listaPunti()` che restituisce tutti i punti di una `QuasiRetta` in un `ArrayList`. Pur trattandosi di un metodo osservatore puro, la sua implementazione potrebbe essere sbagliata, nel senso di consentire l'accesso diretto alla rappresentazione interna. Si descriva una implementazione che ha questo inconveniente.

SOLUZIONE:

```
//@requires p1!= null && p2!=null && approx>=0 && !p1.equals(p2);
//@ensures APPROX=approx && appartiene(p1) && appartiene(p2) &&
//@     dist(p1)==0 && dist(p2)==0 &&
//@    (\forall Punto p; appartiene(p); p1.equals(p) || p2.equals(p));
public QuasiRetta(Punto p1, Punto p2, float approx)
```

Seguono gli altri metodi:

```
//@requires p!= null
//@ensures dist(p)<APPROX && appartiene(p) &&
//@    (\forall Punto p1; !p.equalsd(p1); \old(appartiene(p1)) <==> appartiene(p1));
//@signals(llegalPointException e)  dist(p)>=APPROX/1000  &&
//@    (\forall Punto p1;; \old(appartiene(p1)) <==> appartiene(p1));
public void aggiungi(Punto p)  throws IllegalPointException

//@requires p!= null &&
//@     (\num_of Punto p1; appartiene(p1); !p.equals(p1)) >=2;
//@ensures  !appartiene(p) &&
//@    (\forall Punto p1; !p.equals(p1); \old(appartiene(p1)) <==> appartiene(p1));
public void elimina(Punto p)

//@ensures \result == (\num_of Punto p; appartiene(p); dist(p)<APPROX/1000);
public /*@ pure @*/  int totBuoni()
```

Per l'esportazione della rep, si puo' ipotizzare che QuasiRetta sia memorizzata come un arrayList di punti e che il metodo listaPunti esporti il riferimento all'arraylist.

Esercizio 2

Si consideri una classe Roditore che ha due eredi Ratto e Topo. La classe Topo ha come erede la classe Topolino. Si considerino i casi seguenti:

```
Roditore rod;
Ratto rat = new Ratto();
Topo jerry = new Topo();
Topolino mickey = new Topolino();
```

Segnare con una X gli assegnamenti seguenti che generano un errore di compilazione, e spiegare il perche':

- rod = rat;
- rod = jerry;
- mickey = null;
- mickey =rat;

Soluzione: mickey =rat e' scorretta

Quali delle seguenti dichiarazioni di array sono corrette per un array che ci si attende che possa contenere fino a 10 oggetti dei tipi Ratto, Topo e Topolino? Motivare le risposte in caso positivo e negativo.

1. Ratto[] array = new Ratto[10];
2. Roditore[] array = new Ratto[10];
3. Roditore[] array = new Roditore[10];
4. Roditore[10] array;

Soluzione: Ipotizziamo, a titolo di esempio, che nel codice ci siano i seguenti assegnamenti che inseriscono in array oggetti dei tipi richiesti:

```
array[0]= new Topolino();
array[1] = new  Ratto();
array[2]= new Topo();
```

Caso (1) KO: Ratto non e' compatibile con Topo e Topolino: si avra' errore di compilazione per l'istruzione array[0]= new Topolino().
Caso (2) KO: Il tipo dinamico dell'array diventa Ratto[]. Quindi, durante la compilazione non si avranno errori, ma eseguendo array[0]= new Topolino() si ottiene un errore a runtime (ArrayStoreException), in quanto la chiamata del costruttore new Ratto[10] segnala che l'array ha tido dinamico Ratto, che non e' compatibile con Topo e Topolino. Si noti, infatti, che gli array, a differenza delle collezioni, sono covarianti.
Caso (3) OK, si costruisce un array di tipo Roditore, sovratipo di Ratto, Topo e Topolino.
Caso (4): Sintatticamente scorretto.

Esercizio 3

Si consideri il seguente programma Java

```
public class Esame implements Runnable
{
    private transient int x;
    private volatile int y;

    public static void main(String [] args)
    {
        Esame that = new Esame();
        (new Thread(that)).start( ); /* Linea 8 */
        (new Thread(that)).start( ); /* Linea 9 */
    }
    public synchronized void run( ) /* Linea 11 */
    {
        for (;;) /* Line 13 */
        {
            x++;
            y++;
            System.out.println("x = " + x + " , y = " + y);
        }
    }
}
```

Quali delle seguenti risposte sono corrette? Si fornisca anche una breve spiegazione per la risposta fornita.

1. Si genera un errore di compilazione in corrispondenza di Linea 11;
2. Si genera un errore di compilazione a causa di errori alle Linee 8 e 9;
3. Il programma stampa valori per x e y che potrebbero non essere sempre uguali sulla stessa riga di stampa (per esempio, può stampare $x = 1\ ,\ y = 2$);
4. Il programma stampa valori per x e y che sono sempre gli stessi sulla stessa riga di stampa (per esempio, stampa $x = 1\ ,\ y = 1$ su una riga e $x = 2\ ,\ y = 2$ sulla successiva);
5. Il programma stampa i valori di x in ordine crescente, ovvero
   $x = 1$, ...
   $x = 2$, ...
   $x = 3$, ...

Soluzione: Gli unici casi veri sono (4) e (5). Infatti, il primo thread che entra nel metodo synchronized va in loop infinito e blocca l'accesso all'altro thread: il programma si comporta come se fosse un normale programma sequenziale che non termina. NB: Le dichiarazioni transient e volatile in questo caso non hanno alcun effetto; le variabili d'istanza x e y sono iniziliazzate a 0 per default.

Esercizio 4

Si consideri il seguente frammento di programma (a, b e c sono variabili booleane):

```
if ((a || b) && c)
{
  << Statements >>
}
else
{
   << Statements >>
}
```

Si definisca il minimo insieme di casi di test per ciascuno dei seguenti casi:

1. vengono coperte tutte le istruzioni;
2. vengono coperti tutti i branch;
3. vengono coperti tutti i branch e tutte le condizioni.

Giustificate le vostre risposte.

1) Bastano due casi:

```
A = true  | B = not eval | C = false
A = true  | B = not eval | C = true
```

dove "not eval" significa che la condizione non viene valutata e quindi il valore e' irrilevante.
2) Stessi caso del punto predecente, in quanto vi sono statements per ogni branch.
3) Servono tre casi, a causa della valutazione cortocircuitata di Java.

```
A = true  | B = not eval | C = false
A = false | B = true     | C = true
A = false | B = false    | C = not eval
```

Esercizio 5

Si scriva il risultato dell'esecuzione del seguente frammento di programma:

```
List<Integer> cinqueInteri = Arrays.asList(1, 2, 3, 4, 5);

List<Integer> x = cinqueInteri.stream()
        .map((n) -> {
            return n * 2;
        })
        .collect(Collectors.toList());

System.out.println("x = " + x);

List<Integer> y = cinqueInteri.stream()
        .filter((n) -> n % 2 == 0)
        .collect(Collectors.toList());

System.out.println("y = " + y);

Optional<Integer> z = cinqueInteri.stream()
             .reduce((n, a) -> n + a);

System.out.println("z = " + z.orElse(42));

Optional<Integer> w = new ArrayList<Integer>().stream()
             .reduce((n, a) -> n + a);
     System.out.println("w = " + w.orElse(42));
```

Soluzione:

```
x = [2, 4, 6, 8, 10]
y = [2, 4]
z = 15 (ossia calcola la somma)
w = 42
```

NB: Il programma ha bisogno ovviamente che siano presenti le seguenti dichiarazioni di import:

```
import java.util.stream.Collectors;
import java.util.stream.Stream;
```